

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Частотные методы

Отчёт по лабораторной работе №6

Работу
выполнил:
Сухов Н. М.
Группа: R3235
Преподаватель:
Догадин Е. В.

Санкт-Петербург
2025

Содержание

1. Фильтрация изображений с периодичностью	3
2. Исследование свёртки	5
2.1. Ядра размытия по Гауссу	7
2.2. Ядра блочного размытия	7
2.3. Ядро увеличения резкости	7
2.4. Ядро выделения краёв	7
2.5. Ядро негатива	7
3. Приложение	8

1. Фильтрация изображений с периодичностью

Скачаем изображение 1 с [этого гугл-диска](#) (см. рис. 1.1). Видим периодичность с углом около 40° .

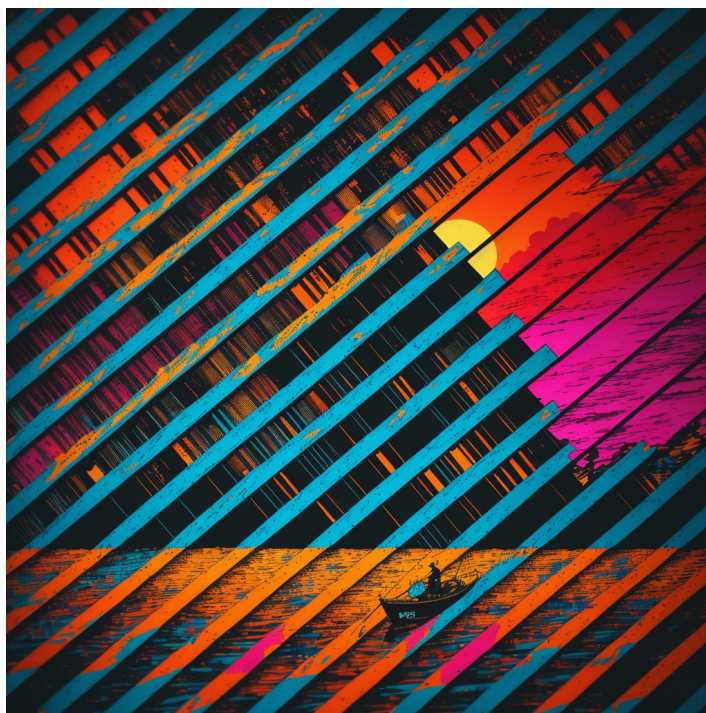


Рисунок 1.1. Картинка

Выполним следующие шаги:

- Загрузим изображение в Python при помощи команды `cv2.imread()`.
- Преобразуем полученный массив к вещественному типу и поделим все значения на 255 - максимальное значение яркости цвета.
- Найдём двумерный Фурье-образ массива и сдвинем его в центр с помощью команд `fft.fftshift(fft.fft2())` для каждого из цветовых каналов.
- Разделим полученный образ на массивы модулей и аргументов с помощью функций `np.abs` и `np.angle`.
- Для удобства работы, найдём логарифм от массива модулей и нормализуем его значения в диапазон от 0 до 1. Чтобы избежать неопределённости в логарифме, предварительно прибавим ко всем значениям 1.
- Сохраним полученный массив (нормализованный логарифм модуля Фурье-образа) как изображение командой `cv2.imwrite`.
- Проанализируем полученное на предыдущем шаге изображение. Найдём пики, соответствующие периодичности на исходной картинке.
- Отредактируем полученный Фурье-образ: сгладим все ненужные цветовые пики, отвечающие за гармоники, от которых хотим избавиться.
- Восстановим картинку из отредактированного образа, проделав обратные шаги.

Изображение логарифма модуля образа исходной картинке представлено на рис. 1.2.

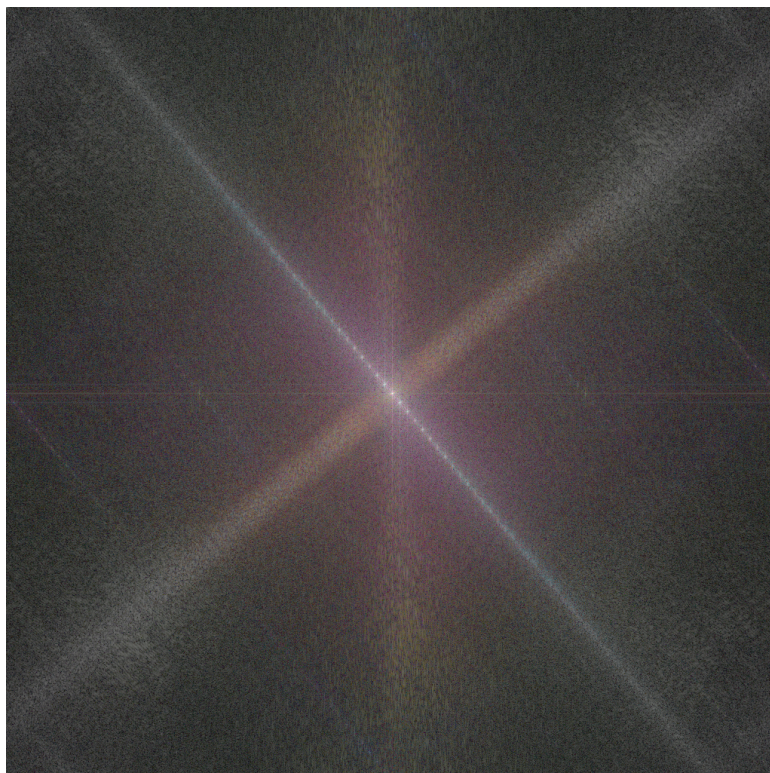
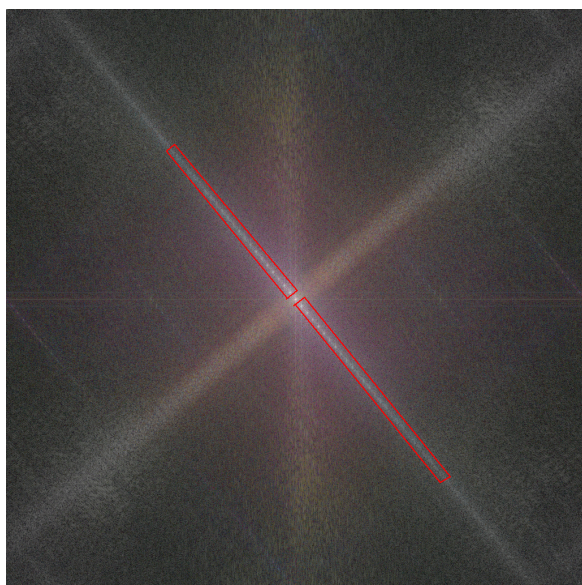
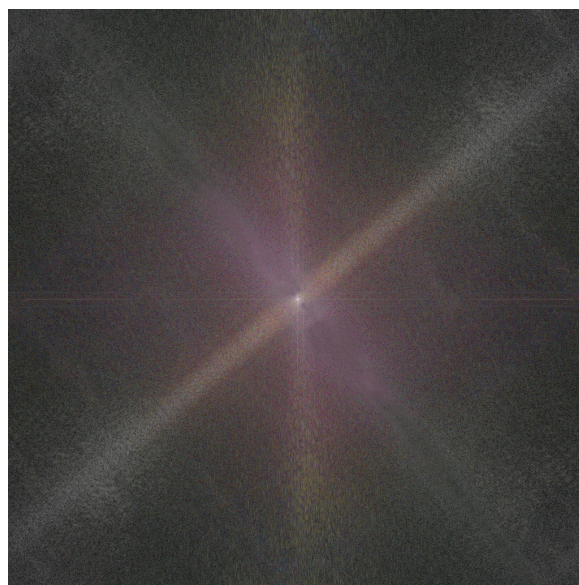


Рисунок 1.2. Логарифм модуля образа

Мы видим множество пиков, расположенных по диагонали на картинке. Выделим их, уберём и посмотрим, что получится (см. рис.1.3).



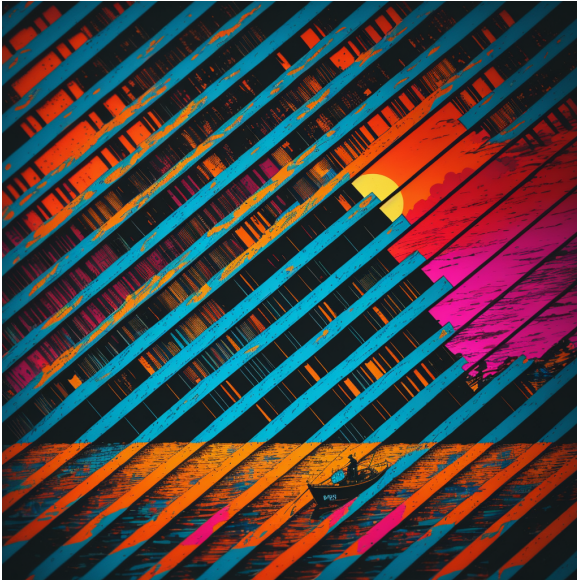
(a)



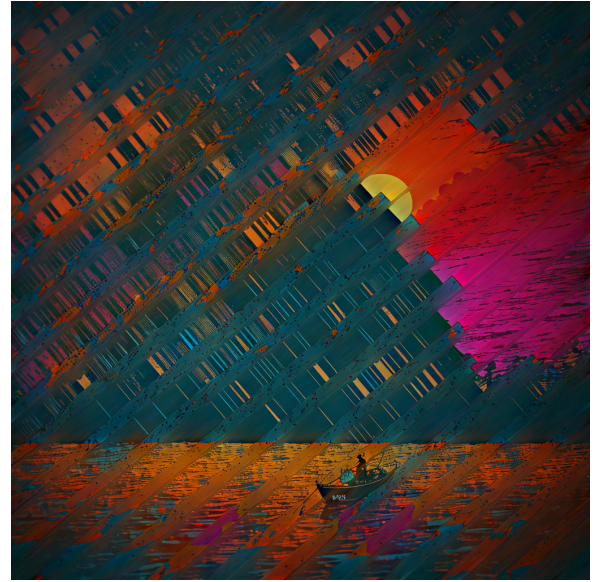
(b)

Рисунок 1.3. Выделенные области с пиками на спектре (a) и очищенный спектр (b)

Сравним теперь исходное и отфильтрованное изображение (см. рис. 1.4). Видим, что периодичность исчезла почти полностью, но есть и побочный эффект - изображение стало более тусклым. Это происходит потому что, затирая пики, мы задеваем часть остальной информации изображения, от чего оно искажается.



(a)



(b)

Рисунок 1.4. Исходное изображение (a) и отфильтрованное (b)

Выводы

Используя многомерное преобразование Фурье, можно осуществлять фильтрацию на изображениях, в данном случае, мы видим, как можно убирать гармоники из спектра, создающие периодические структуры на изображении. При этом чем сильнее мы гасим пики на спектре, тем больше информации мы задеваем, и изображение немного искажается (меняются оттенки), поэтому нельзя затирать их слишком сильно, нужно искать "золотую середину", когда периодичность убрана достаточно хорошо, и при этом нет сильного изменения цветов на изображении, чтобы оно не стало совсем тусклым.

2. Исследование свёртки

Выберем изображение не очень большого размера (см. рис. 2.1a). Преобразуем его в ЧБ (см. рис. 2.1b).

Выберем ядра свёртки:

- Ядра размытия по Гауссу для трёх показательных нечётных значений $N \geq 3$ (выберем N из 3, 7, 15):

$$\sigma = \frac{N-1}{6}, \quad A_{ij} = \exp\left(-\frac{(i - \frac{N+1}{2})^2 + (j - \frac{N+1}{2})^2}{2\sigma^2}\right), \quad i, j \in \{1, \dots, N\},$$

$$K_\sigma = \frac{A}{\sum_{i,j} A_{i,j}}, \quad \text{где } A - \text{вся матрица}$$

- Ядра блочного размытия для тех же значений N :

$$A_{ij} = 1, \quad i, j \in \{1, \dots, N\}, \quad K_\square = \frac{A}{\sum_{i,j} A_{i,j}}$$

- Ядро увеличения резкости:

$$K_* = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- Ядро выделения краёв:

$$K_{\nabla} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- Ядро негатива:

$$K_{neg} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$



(a)



(b)

Рисунок 2.1. Выбранное изображение (a) и ЧБ (b)

Логарифм модуля образа исходной картинки представлен на рис. [2.2](#).



Рисунок 2.2. Логарифм модуля образа

- 2.1. Ядра размытия по Гауссу
- 2.2. Ядра блочного размытия
- 2.3. Ядро увеличения резкости
- 2.4. Ядро выделения краёв
- 2.5. Ядро негатива

3. Приложение

```
1  # %%
2  import numpy as np
3  import cv2
4  from scipy import fft
5  import matplotlib.pyplot as plt
6  from scipy.signal import convolve2d
7
8  # %%
9  plt_folder = "plots/"
10
11  # %% [markdown]
12  # # Задание 1
13
14  # %%
15  image_float = cv2.imread('1.png').astype(np.float64) / 255.0
16
17  fft_channels = []
18  for channel in range(3):
19      channel_data = image_float[:, :, channel]
20      fft_channel = fft.fftshift(fft.fft2(channel_data))
21      fft_channels.append(fft_channel)
22  fft_image = np.stack(fft_channels, axis=-1)
23
24  original_fft = fft_image.copy()
25
26  magnitude_spectrum = np.abs(fft_image)
27  phase_spectrum = np.angle(fft_image)
28
29  log_magnitude = np.log1p(magnitude_spectrum)
30  normalized_log = (log_magnitude - np.min(log_magnitude)) /
31  ↪ (np.max(log_magnitude) - np.min(log_magnitude))
32
33  spectrum_image = (normalized_log * 255).astype(np.uint8)
34  cv2.imwrite(plt_folder+'fourier_spectrum_1.png', spectrum_image)
35
36  # %%
37  spectrum_filtered = cv2.imread(plt_folder +
38  ↪ 'fourier_spectrum_1_filtered.jpg').astype(np.float64) / 255.0
39
40  log_magnitude_filtered = spectrum_filtered * (np.max(log_magnitude) -
41  ↪ np.min(log_magnitude)) + np.min(log_magnitude)
42  magnitude_filtered = np.expm1(log_magnitude_filtered)
43
44  filtered_fft_channels = []
45  for channel in range(3):
46      magnitude_channel = magnitude_filtered[:, :, channel]
47      phase_channel = phase_spectrum[:, :, channel]
48      complex_spectrum = magnitude_channel * np.exp(1j * phase_channel)
49      ↪ filtered_fft_channels.append(fft.ifft2(fft.ifftshift(complex_spectrum)))
50
51  reconstructed_image = np.stack(filtered_fft_channels, axis=-1)
52  reconstructed_image = np.real(reconstructed_image)
53  reconstructed_image = np.clip(reconstructed_image, 0, 1)
54  reconstructed_image_uint8 = (reconstructed_image * 255).astype(np.uint8)
55
56  cv2.imwrite(plt_folder + 'periodic_filtered_image.png',
57  ↪ reconstructed_image_uint8)
58
59  # %% [markdown]
60  # # Задание 2
61
62  # %%
63  dogsimg = cv2.imread("dogs.jpg")
64  gray_img = cv2.cvtColor(dogsimg, cv2.COLOR_BGR2GRAY)
65  cv2.imwrite("black_dogs.jpg", gray_img)
66  img = np.array(gray_img)
67  h, w = img.shape
68  N_values = [7, 23, 47]
```



```

66 # %% [markdown]
67 # ##### Ядра
68
69 # %%
70 def gaussian_kernel(N):
71     sigma = (N - 1) / 6
72     center = (N + 1) / 2
73     i = np.arange(1, N+1) - center
74     j = np.arange(1, N+1) - center
75     ii, jj = np.meshgrid(i, j)
76     kernel = np.exp(-(ii**2 + jj**2) / (2 * sigma**2))
77     return kernel / np.sum(kernel)
78
79 def box_kernel(N):
80     kernel = np.ones((N, N))
81     return kernel / np.sum(kernel)
82
83 sharp_kernel = np.array([[0, -1, 0],
84                           [-1, 5, -1],
85                           [0, -1, 0]])
86
87 edge_kernel = np.array([[-1, -1, -1],
88                          [-1, 8, -1],
89                          [-1, -1, -1]])
90
91 negative_kernel = np.array([[0, 0, 0],
92                             [0, -1, 0],
93                             [0, 0, 0]])
94
95 # %%
96 fft_original = fft.fftshift(fft.fft2(img))
97 log_magnitude_original = np.log1p(np.abs(fft_original))
98
99 # %%
100 #туту надо сделать "Приведено изображение логарифма модуля образа
    ↳ исходной картинки."
101
102 # %%
103 plt.rcParams['axes.titlesize'] = 10
104
105 # %% [markdown]
106 # ##### Размытие по Гауссу
107
108 # %%
109 fig = plt.figure(figsize=(15, 18))
110
111 plt.subplot(3, 3, 1)
112 plt.axis('off')
113
114 plt.subplot(3, 3, 2)
115 plt.imshow(img, cmap='gray')
116 plt.title('Исходное изображение')
117 plt.axis('off')
118
119 plt.subplot(3, 3, 3)
120 plt.axis('off')
121
122 for i, N in enumerate(N_values):
123     kernel = gaussian_kernel(N)
124
125     conv_result = convolve2d(img, kernel, mode='same', boundary='symm')
126     conv_result = np.clip(conv_result, 0, 255)
127
128     k_h, k_w = kernel.shape
129     fft_size = (h + k_h - 1, w + k_w - 1)
130     fft_img_padded = fft.fft2(img, s=fft_size)
131     fft_kernel = fft.fft2(kernel, s=fft_size)
132     result_fft = fft.ifft2(fft_img_padded * fft_kernel)
133
134     start_i = (fft_size[0] - h) // 2
135     start_j = (fft_size[1] - w) // 2
136     fft_result = np.real(result_fft[start_i:start_i+h,
    ↳ start_j:start_j+w])
137     fft_result = np.clip(fft_result, 0, 255)
138

```

```

139 plt.subplot(3, 3, i+4)
140 plt.imshow(conv_result, cmap='gray')
141 plt.title(f'Обычная свертка: N={N}')
142 plt.axis('off')
143
144 plt.subplot(3, 3, i+7)
145 plt.imshow(fft_result, cmap='gray')
146 plt.title(f'Фурье: N={N}')
147 plt.axis('off')
148
149 plt.tight_layout()
150 plt.savefig(plt_folder + 'gaussian_comp.png', dpi=300,
151             ↳ bbox_inches='tight')
152 plt.show()
153
154 # %%
155 fig = plt.figure(figsize=(15, 18))
156
157 plt.subplot(3, 1, 1)
158 plt.imshow(log_magnitude_original, cmap='gray')
159 plt.title('Логарифм модуля фурье-образа исходного изображения')
160 plt.axis('off')
161
162 for i, N in enumerate(N_values):
163     kernel = gaussian_kernel(N)
164
165     conv_result = convolve2d(img, kernel, mode='same', boundary='symm')
166     conv_result = np.clip(conv_result, 0, 255)
167
168     k_h, k_w = kernel.shape
169     fft_size = (h + k_h - 1, w + k_w - 1)
170     fft_img_padded = fft.fft2(img, s=fft_size)
171     fft_kernel = fft.fft2(kernel, s=fft_size)
172     result_fft = fft.ifft2(fft_img_padded * fft_kernel)
173
174     start_i = (fft_size[0] - h) // 2
175     start_j = (fft_size[1] - w) // 2
176     fft_result = np.real(result_fft[start_i:start_i+h,
177                                  ↳ start_j:start_j+w])
178     fft_result = np.clip(fft_result, 0, 255)
179
180     fft_conv_spectrum = fft.fftshift(fft.fft2(conv_result))
181     log_magnitude_conv = np.log1p(np.abs(fft_conv_spectrum))
182
183     fft_fft_spectrum = fft.fftshift(fft.fft2(fft_result))
184     log_magnitude_fft = np.log1p(np.abs(fft_fft_spectrum))
185
186     plt.subplot(3, 3, i+4)
187     plt.imshow(log_magnitude_conv, cmap='gray')
188     plt.title(f'Обычная свертка: N={N}')
189     plt.axis('off')
190
191     plt.subplot(3, 3, i+7)
192     plt.imshow(log_magnitude_fft, cmap='gray')
193     plt.title(f'Фурье: N={N}')
194     plt.axis('off')
195
196 plt.tight_layout()
197 plt.savefig(plt_folder + 'gaussian_log.png', dpi=300,
198             ↳ bbox_inches='tight')
199 plt.show()
200
201 # %% [markdown]
202 # ##### Блочное размытие
203
204 # %%
205
206 # %% [markdown]
207 # ##### Увеличение резкости
208
209 # %%
210 kernel = sharp_kernel
211 conv_result = convolve2d(img, kernel, mode='same', boundary='symm')

```

```

211 conv_result = np.clip(conv_result, 0, 255)
212
213 k_h, k_w = kernel.shape
214 fft_size = (h + k_h - 1, w + k_w - 1)
215 fft_img_padded = fft.fft2(img, s=fft_size)
216 fft_kernel = fft.fft2(kernel, s=fft_size)
217 result_fft = fft.ifft2(fft_img_padded * fft_kernel)
218
219 start_i = (fft_size[0] - h) // 2
220 start_j = (fft_size[1] - w) // 2
221 fft_result = np.real(result_fft[start_i:start_i+h, start_j:start_j+w])
222 fft_result = np.clip(fft_result, 0, 255)
223
224 fft_kernel_shifted = fft.fftshift(fft_kernel)
225 log_magnitude_kernel = np.log1p(np.abs(fft_kernel_shifted))
226
227 fft_result_spectrum = fft.fftshift(fft.fft2(fft_result))
228 log_magnitude_result = np.log1p(np.abs(fft_result_spectrum))
229
230 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
231
232 axes[0, 0].imshow(img, cmap='gray')
233 axes[0, 0].set_title('Исходное изображение')
234 axes[0, 0].axis('off')
235
236 axes[0, 1].imshow(conv_result, cmap='gray')
237 axes[0, 1].set_title('Обычная свертка: Увеличение резкости')
238 axes[0, 1].axis('off')
239
240 axes[0, 2].imshow(fft_result, cmap='gray')
241 axes[0, 2].set_title('Фурье: Увеличение резкости')
242 axes[0, 2].axis('off')
243
244 axes[1, 0].imshow(log_magnitude_original, cmap='gray')
245 axes[1,
    ↪ 0].set_title('Логарифм модуля фурье-образа исходного изображения')
246 axes[1, 0].axis('off')
247
248 axes[1, 1].imshow(log_magnitude_kernel, cmap='gray')
249 axes[1, 1].set_title('Логарифм модуля фурье-образа ядра')
250 axes[1, 1].axis('off')
251
252 axes[1, 2].imshow(log_magnitude_result, cmap='gray')
253 axes[1, 2].set_title('Логарифм модуля фурье-образа результата')
254 axes[1, 2].axis('off')
255
256 plt.tight_layout()
257 plt.savefig(plt_folder + 'sharp_comp.png', dpi=300, bbox_inches='tight')
258 plt.show()
259
260
261 # %% [markdown]
262 # ##### Выделение краёв
263
264 # %%
265
266
267 # %% [markdown]
268 # ##### Негатив
269
270 # %%
271
272
273 # %%
274
275
276

```

Листинг 1: Листинг